

Regression algorithms

Marzo 12 de 2019 – Marzo 19 de 2019

Carlos Alejandro Trujillo Anaya – carlostrujillo@itm.edu.co

M212

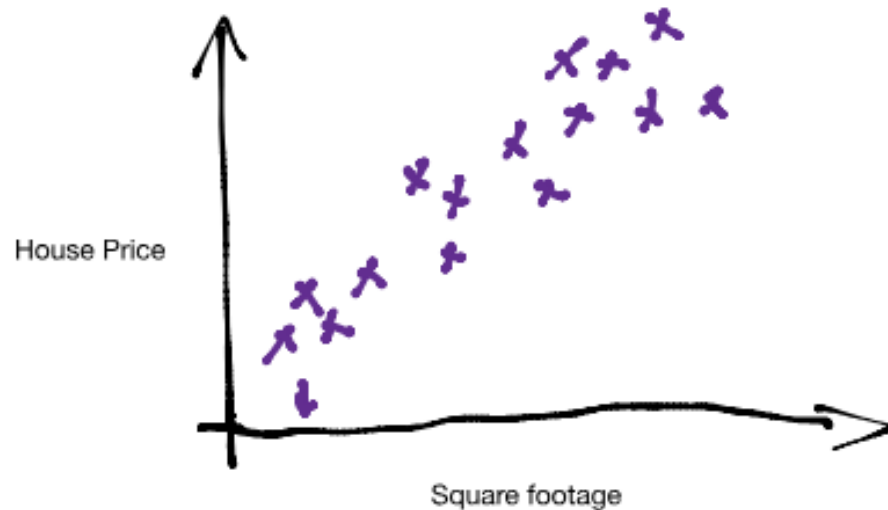
Electronics and telecommunications department

Course belonging to “Maestría en Automatización y Control Industrial”.

Tuesday 18-21

Linear regression

Regression differs from classification in that the output or predicted feature in regression problems is continuous.



In this case... we can define
a pattern!

We want our algorithm to
find and use this pattern to
predict house prices based
on house size.

<https://www.kdnuggets.com/2017/05/machine-learning-crash-course-part-1.html>

Parametric vs. non-parametric regression

The most common approach in regression is to fit the data to a global parametric function.

Examples are linear and polynomial regression!

The fitting criterion is generally a minimization of an error function for all data points in a training set.

Very often, this is a ***least squares*** criterion, which minimizes the sum of the squares of the prediction errors of the estimated linear function for all instances.

$$\text{RMSE} = \sqrt{\frac{\sum_{t=1}^T (\hat{y}_t - y_t)^2}{T}}$$

$\hat{y}_t$ is the predicted value

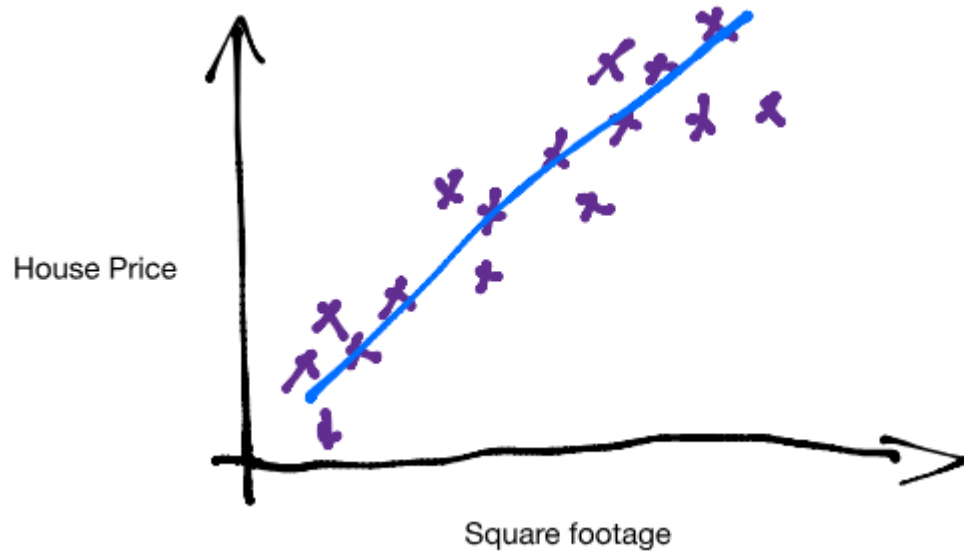
Linear regression



Intuitively we can see that there is a diagonal strip in the graph that most houses seem to land on

<https://www.kdnuggets.com/2017/05/machine-learning-crash-course-part-1.html>

Linear regression



The blue line, called a predictor, predicts the price of a house from its square footage.

So we've seen examples of one and two input variables, but many machine learning applications take into account hundreds and even thousands of variables.

<https://www.kdnuggets.com/2017/05/machine-learning-crash-course-part-1.html>

Linear regression

The predictor

$$f(x) = c_0 + c_1x_1 + \dots + c_nx_n$$

x is a feature

c is a parameter or weight

*In our example, the parameters could be the square footage
and the number of power outlets!*

<https://www.kdnuggets.com/2017/05/machine-learning-crash-course-part-1.html>

Linear regression

The predictor (in our example with only two variables)

$$y(x) = c_1x + c_0$$

y(x): price of the house

x: size of the house

c₀: the base price of the house

The goal is to calculate the parameters!
The classical methods can get impractical when working with hundreds or thousands of variables, so we use machine learning algorithms.

<https://www.kdnuggets.com/2017/05/machine-learning-crash-course-part-1.html>

Polynomial regression

In statistics, polynomial regression is a form of regression analysis in which the relationship between the independent variable x and the dependent variable y is modelled as an n th degree polynomial in x .

Polynomial
Linear
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

Advantages:

Polynomials fit a wide range of curvature.

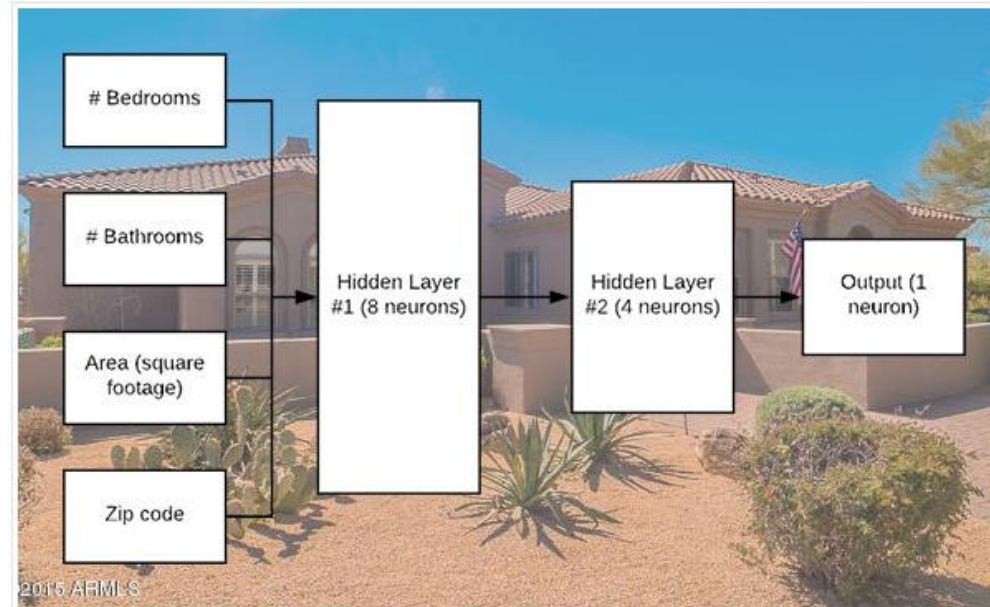
Polynomials provide a good approximation of the relationship.

Disadvantages:

Strong sensitivity to outliers!

Parametric vs. non-parametric regression

In machine learning, parametric learning methods have found wide application areas in the form of simple **neural networks (Perceptron)**



```
# check to see if the regression node should be added
if regress:
    model.add(Dense(1, activation="linear"))
```

<https://www.pyimagesearch.com/2019/01/21/regression-with-keras/>

Parametric vs. non-parametric regression

Benefits of Parametric Machine Learning Algorithms

Simpler: These methods are easier to understand and interpret results.

Speed: Parametric models are very fast to learn from data.

Less data: They do not require as much training data and can work well even if the fit to the data is not perfect.

Limitations of Parametric Machine Learning Algorithms

Constrained: By choosing a functional form these methods are highly constrained to the specified form.

Limited Complexity: The methods are more suited to simpler problems.

Poor Fit: In practice the methods are unlikely to match the underlying mapping function.

Parametric vs. non-parametric regression

Algorithms that do not make strong assumptions about the form of the mapping function are called ***nonparametric machine learning algorithms***.

By not making assumptions, they are free to learn any functional form from the training data.

Support Vector Machines
Convolutional neural networks

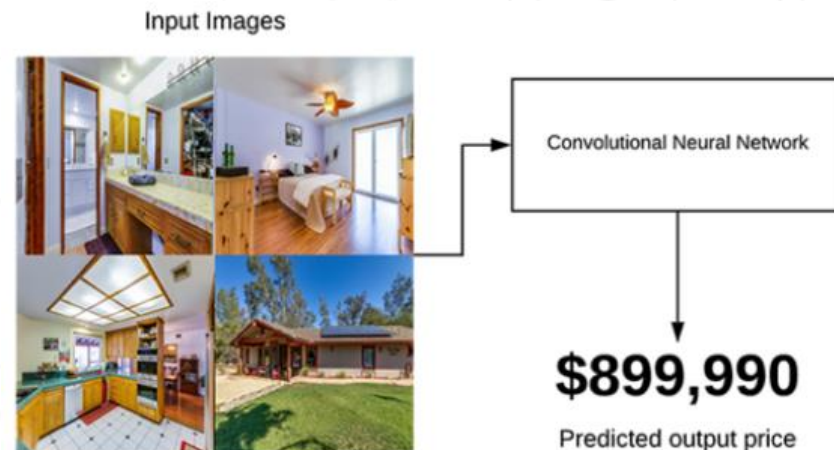
Parametric vs. non-parametric regression

Algorithms that do not make strong assumptions about the form of the mapping function are called ***nonparametric machine learning algorithms***.

By not making assumptions, they are free to learn any functional form from the training data.

Support Vector Machines for regression
Convolutional neural networks

<https://www.pyimagesearch.com/2019/01/28/keras-regression-and-cnns/>



Instance-based regression

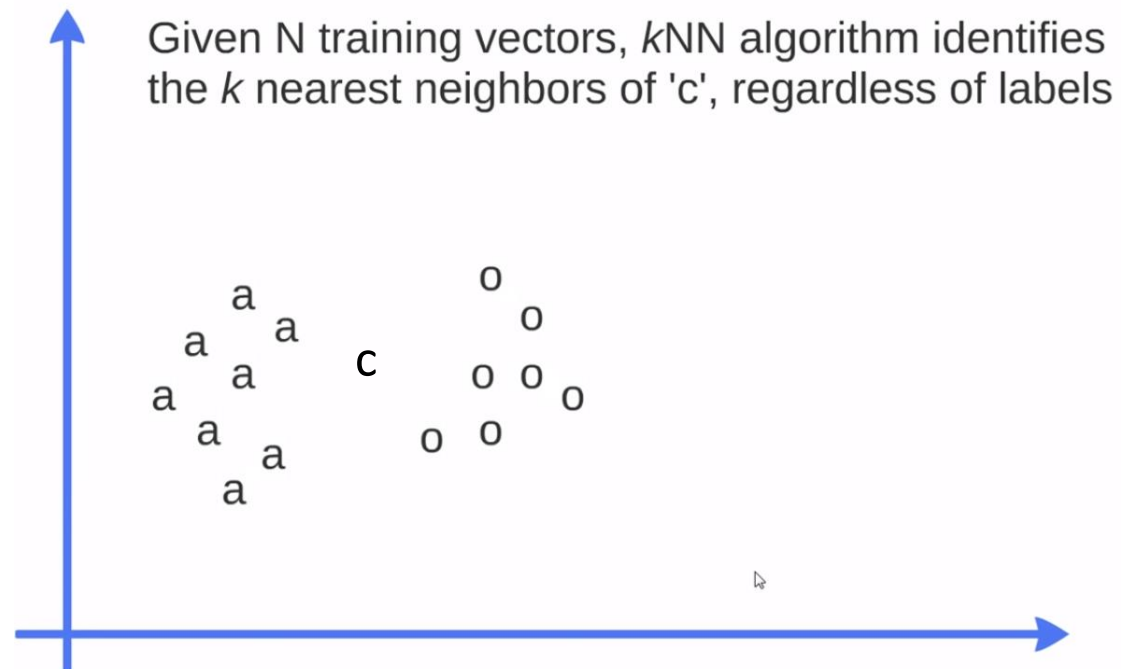
In instance-based regression, each instance is usually represented as a set of attribute value pairs, where the values are either nominal or numeric, and the value to be predicted is continuous.

In most applications, training is done simply by storing the instances.

The problem to be solved is, for a given query instance, to predict the target value as a function of other instances whose target values are known.

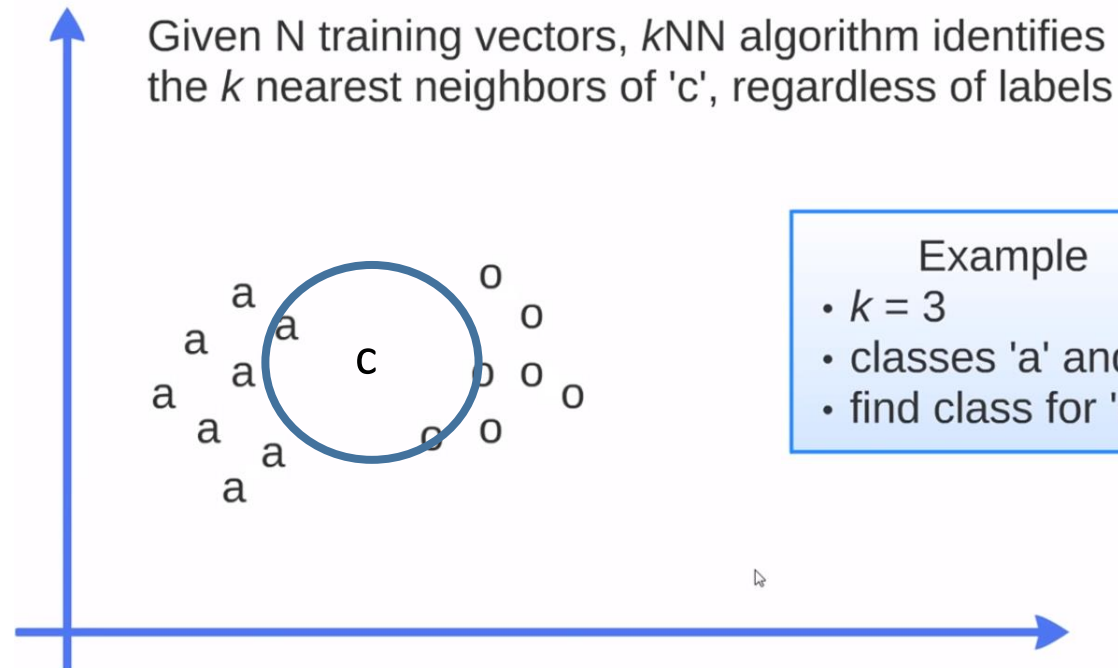
Instance-based regression

k-NN (k-nearest neighbor)



Instance-based regression

k-NN (k-nearest neighbor)



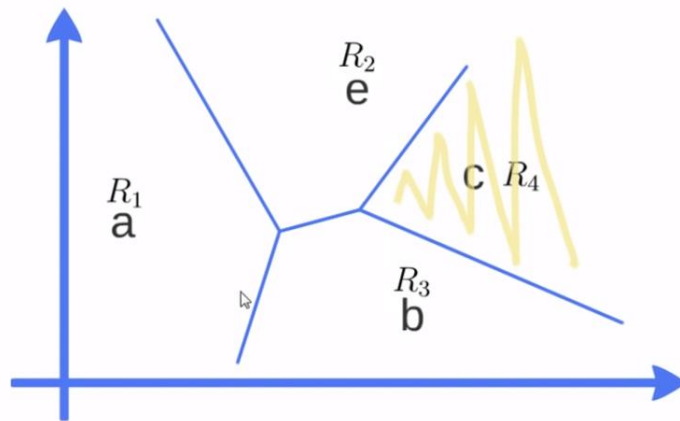
Example

- $k = 3$
- classes 'a' and 'o'
- find class for 'c'

Instance-based regression

k-NN (k-nearest neighbor)

When $k = 1$, each training vector defines a region in space, defining a *Voronoi* partition of the space



$$R_i = \{x : d(x, x_i) < d(x, x_j), i \neq j\}$$

Remarks

- chose an odd k value for a 2 class problem
- k must not be a multiple of the number of classes
- the main drawback of k NN is the complexity in searching the nearest neighbors for each sample

Instance-based regression

k-NN (k-nearest neighbor)

Formally, if we let real numbers, R , be a numeric domain, and X be an instance space with p attributes, we can then describe the approximation function, F , for predicting numeric values as follows:

$$F(x_1, \dots, x_p) = \bar{y}_i \quad \text{where } \bar{y}_i \in R$$

Instance-based regression

The proximity algorithm

$$\text{Similarity}(\mathbf{x}_t, \mathbf{x}_i) = \sum_{j=1}^p \text{Sim}(x_{tj}, x_{ij})$$

$$\text{Sim}(x, y) = 1.0 - |x - y|$$

- [1] $\forall \mathbf{x}_i \in \text{Training Set}$
- [2] $\text{normalize}(\mathbf{x}_i)$
- [3] $\forall \mathbf{x}_t \text{ Test Set}$
- [4] $\text{normalize}(\mathbf{x}_t)$
- [5] $\forall \mathbf{x}_i \{ \mathbf{x}_i \neq \mathbf{x}_t \}$: Calculate $\text{Similarity}(\mathbf{x}_t, \mathbf{x}_i)$
- [6] Let *Similar* be set of N most similar instances to $\mathbf{x}_t$ in Training Set
- [7] Let $\text{Sum} = \sum_{\mathbf{x}_i \in \text{Similar}} \text{Similarity}(\mathbf{x}_t, \mathbf{x}_i)$
- [8] Then $\bar{y}_t = \sum_{\mathbf{x}_i \in \text{Similar}} \frac{\text{Similarity}(\mathbf{x}_t, \mathbf{x}_i)}{\text{Sum}} F(\mathbf{x}_i)$

The Knowledge Engineering Review, Vol. 14:4, 1999, 319-340 (KER 14404)

Instance-based regression

For sufficiently large sample sizes, this technique yields a good approximation for continuous functions.

By default, instance-based regression assumes that all the features are equivalently relevant

The most important drawback of instance-based algorithms is that they do not yield abstractions or models that enable interpretation of the training sets

Locally weighted regression

This methods are similar to IBR due to three properties:

- The training phases of both algorithms include just storing the training data, and the main work is done during prediction.
- Both kind of algorithms predict query instances by strongly influencing nearby or similar training instances.
- They represent instances as real-valued points in p -dimensional Euclidean space.

IBR and LWR are also known as ***lazy learning*** methods!

Locally weighted regression

The difference relies on **LWR** makes predictions by forming an averaging model at the location of query instance.

This local model is generally a linear or nonlinear parametric function.

Locally weighted regression

Procedure:

Target function is approximated using a linear function:

$$\hat{f}(x) = w_0 + w_1 a_1(x) + \dots + w_n a_n(x)$$

Methods like gradient descent can be used to calculate the coefficients w_i to minimize the error in fitting such linear functions.

The error function has to be redefined

Locally weighted regression

Possibilities to redefine the error criterion E

1. Minimize the squared error over just the k nearest neighbors.

$$E_1(x_q) \equiv \frac{1}{2} \sum_{x \in k \text{ nearest neighbors}} (f(x) - \hat{f}(x))^2$$

Locally weighted regression

Possibilities to redefine the error criterion E

2. Minimize the squared error over the entire set D , while weighting the error of each training sample by some decreasing function K of its distance from x_q

$$E_2(x_q) \equiv \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2 \cdot K(d(x_q, x))$$

Locally weighted regression

Possibilities to redefine the error criterion E

3. Combine **E1** and **E2**

$$E_3(x_q) \equiv \frac{1}{2} \sum_{x \in k \text{ nearest neighbors}} (f(x) - \hat{f}(x))^2 \cdot K(d(x_q, x))$$

Locally weighted regression

Choice of the error criterion

E_2 is the most esthetically criterion, because it allows every training example to have impact on the classification of **xq** .

However, computational effort grows with the number of training samples!

Remarks on locally weighted linear regression:

- In most cases, constant, linear or quadratic functions are used.
- Costs for fitting more complex functions are prohibitively high.
- Simple approximations are good enough over a sufficiently small sub-region of X

Regression by tree induction

Tree induction algorithms construct the model by partitioning the data set.

E_2 is the most esthetically criterion, because it allows every training example to have impact on the

Regression by tree induction

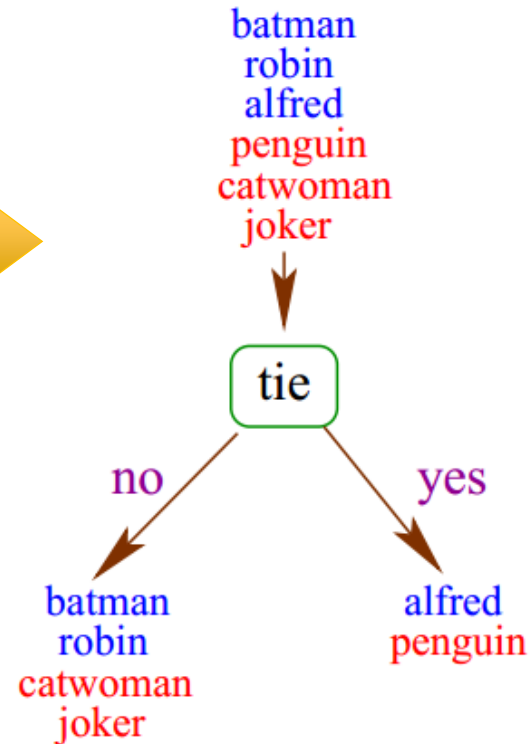
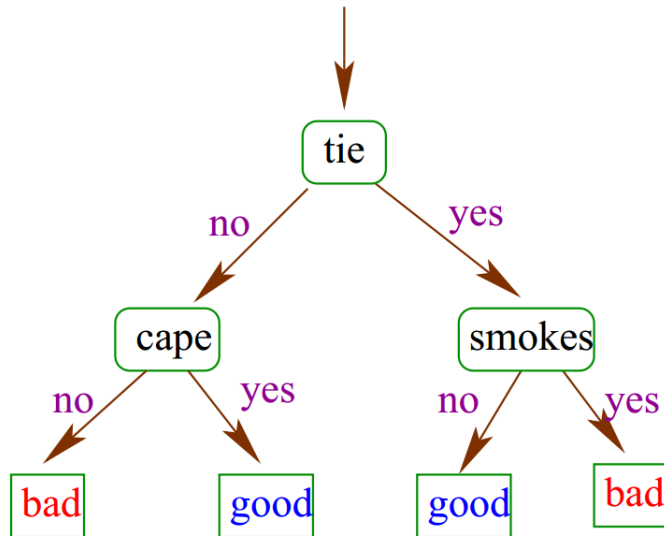
	sex	mask	cape	tie	ears	smokes	class
training data							
batman	male	yes	yes	no	yes	no	Good
robin	male	yes	yes	no	no	no	Good
alfred	male	no	no	yes	no	no	Good
penguin	male	no	no	yes	no	yes	Bad
catwoman	female	yes	no	no	yes	no	Bad
joker	male	no	no	no	no	no	Bad
test data							
batgirl	female	yes	yes	no	yes	no	??
riddler	male	yes	no	no	no	no	??

Problem: Identify the characters as good and bad according to their appearance.

Rob Schapire, Princeton University .

Regression by tree induction

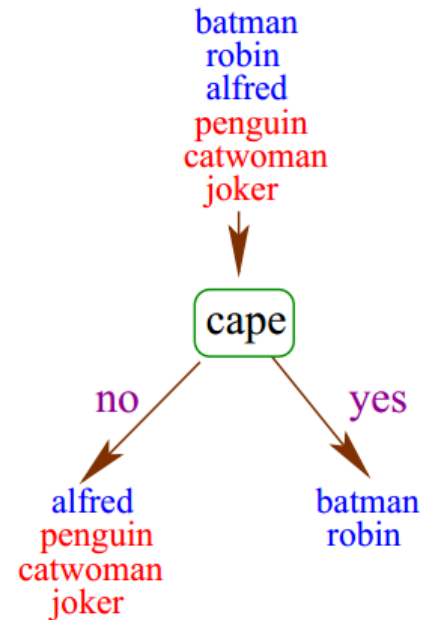
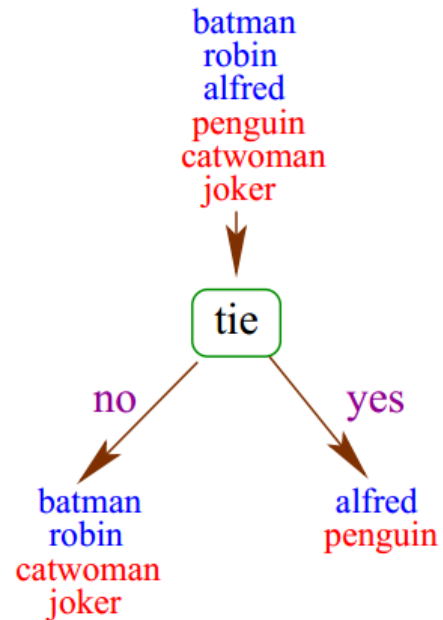
Select a rule to divide the dataset in separable sub-sets.



The idea is to obtain pure sub-sets.

Regression by tree induction

The key problem is to **properly** choose the division rule.

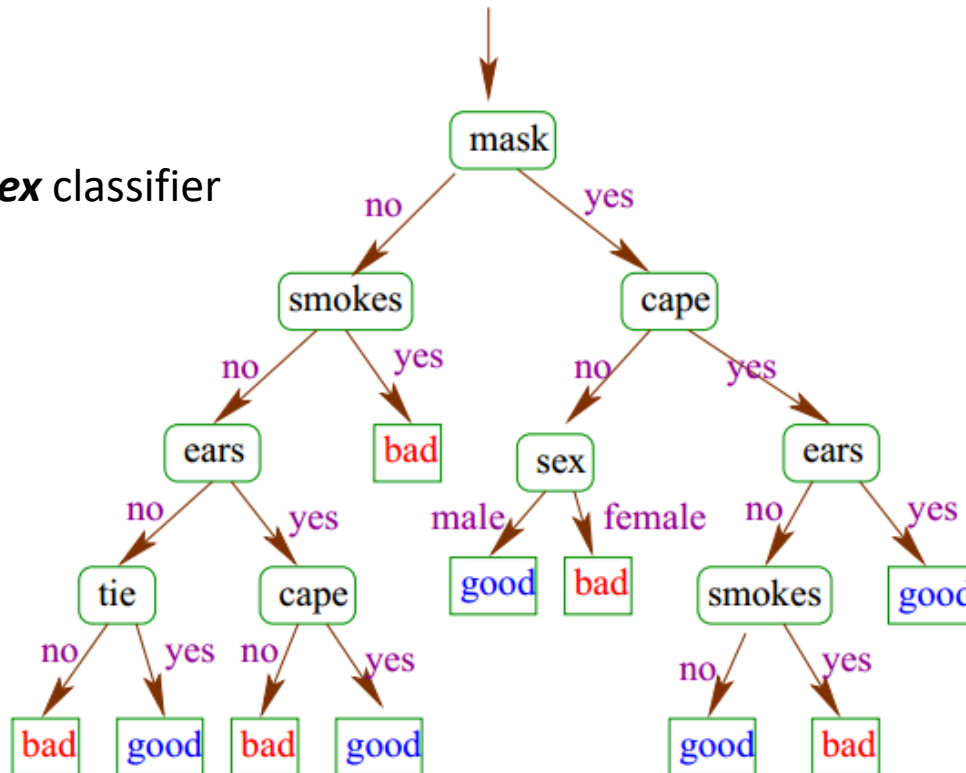


This rule must be chosen considering the **purity** of the subsets.

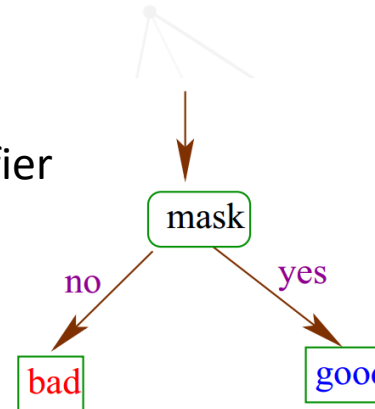
Regression by tree induction

A possible classifier could be one of the following:

Complex classifier

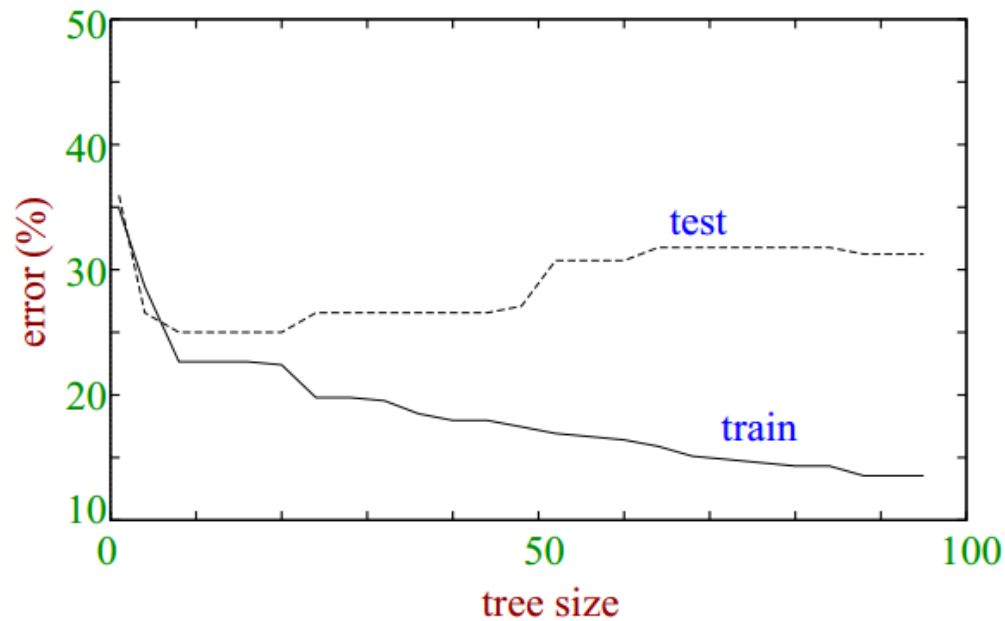


Simple classifier



Regression by tree induction

Tree size vs. classification error



The size of the tree should be large enough to fit the data, but not large enough to **overfit**.

Regression by tree induction

Instead of predicting a discrete outcome in Regression the outcome is continuous.

There's 2 ways to deal with this:

1. Discretize numeric outcome and use standard classification algorithms.
 - Problem: number of classes has to be known in advance.
 - Algorithm used to discretize: **P-Class**.
2. Adapt the algorithm to Regression tasks.
 - Either predict a certain value (Median or Mean) in the head of the rule directly.
 - Use a (linear) model in the head to predict the value (algorithm M5-Rules).

Regression by tree induction

The explanatory capability of regression trees and their use to determine key features from a large feature set are major advantages for these applications.

Regression trees are also noted to be strong when there are higher order dependencies among the predictors.

To avoid **overfitting** and form simpler models, pruning strategies are employed in all regression tree methods.

Regression by tree induction

CART (Classification and Regression Trees)

Procedure:

- 1) We start with the whole training set represented by the root node to construct the tree.
- 2) A search is done on the features to construct the remaining part of the tree recursively.
- 3) The best feature and the feature value are found to split the training set represented by the root node.

Regression by tree induction

CART (Classification and Regression Trees)

Procedure:

- 4) This splitting forms two leaf nodes that represent two disjoint regions in the training set.
- 5) Later, one of these regions is selected for further splitting. This splitting is again done according to a selected feature value.
- 6) This splitting process continue at a selected leaf node recursively to construct the regression tree.

Regression by tree induction

CART (Classification and Regression Trees)

In the CART system, a constant function (a constant response value) is used for estimation of a query or test instance that falls into the regions represented by leaf nodes.

Generally, this function is the average of the response values of instances.

Each disjoint region has its own estimated value that is assigned to any query instance located in this region.

$$\text{If } \mathbf{x} \in R_m, \text{ then } f(\mathbf{x}) = g_m(\mathbf{x} \{a_j\}_1^P)$$

$\{R_m\}_1^P$ are disjoint sub-regions representing p partitions of the training set.

$$g_m(\mathbf{x}|a_m) = a_m$$

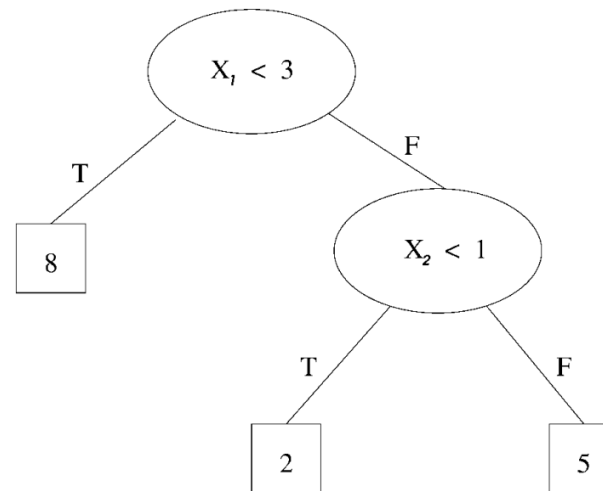
Regression by tree induction

CART (Classification and Regression Trees)

If $\mathbf{x} \in R_m$, then $f(\mathbf{x}) = g_m(\mathbf{x}\{a_j\}_1^P)$

$\{R_m\}_1^P$ are disjoint sub-regions representing p partitions of the training set.

$$g_m(\mathbf{x}|a_m) = a_m$$



Regression by tree induction

Other algorithms

RETIS (Regression Tree Induction System)

In the basic CART, the estimated response value on the leaves of the regression tree was a constant function, on the other hand, in RETIS employs a function on the leaves.

Hence... local linear regression! (other LWR)

Regression by tree induction

Other algorithms

M5

The advantage of M5 over CART is that the trees are generally much smaller than regression trees.

On the other hand, the tree construction in ***M5*** is similar to ***CART***. Standard deviation is employed as the error criterion in ***M5***, instead of variance as used in ***CART***.

$$error = \sigma(\mathbf{X}) - \sum_i \frac{|\mathbf{X}_i|}{|\mathbf{X}|} \sigma(\mathbf{X}_i)$$

After examining all possible splits, ***M5*** chooses the one that maximizes the expected error reduction.

“An overview of regression techniques for knowledge Discovery”, The Knowledge Engineering Review, Vol. 14:4, 1999, 319-340

Regression by rule induction

Both tree and rule induction models find solutions in disjunctive normal form (DNF).

The following formulas are in DNF:

$$(A \wedge \neg B \wedge \neg C) \vee (\neg D \wedge E \wedge F)$$

$$(A \wedge B) \vee C$$

$$A \wedge B$$

$$A$$

The following formulas are **NOT** in DNF:

$$\neg(A \vee B), \text{ since an OR is nested within a NOT}$$

$$A \vee (B \wedge (C \vee D)), \text{ since an OR is nested within an AND}$$

Regression by rule induction

The latter implies that the following formula is applicable for both:

$$\textit{if } \mathbf{x} \subseteq R_i \textit{ then } f(\mathbf{x}) = k_i = \textit{median}\{y_j^i\}$$

$\mathbf{x}$ is an instance

R_i is a rule (or condition)

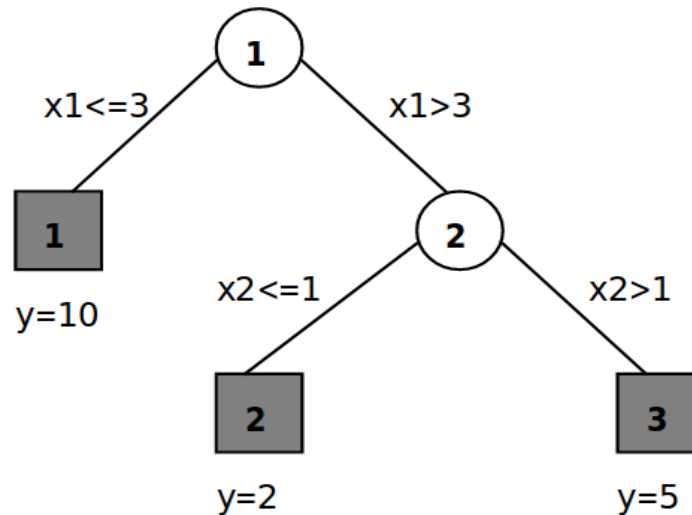
$f(\mathbf{x})$ is the output of the predicting model

k_i is an output class

y_j are the instances in the class k_i

Regression by rule induction

The most important difference between trees and rules induction is that the rules are not mutually exclusive, as in decision trees. In decision trees, for each instance there is exactly one rule, a path from a root to a leaf, that is satisfied.



$x1 \leq 3 \rightarrow y=10$
 $x2 \leq 1 \rightarrow y=2$
Otherwise $y=5$

Regression by rule induction

Some mechanism is needed to resolve the conflicts in k_i when multiple rules are invoked

if $i < j$ and $\mathbf{x} \subseteq$ both R_i and R_j then $f(\mathbf{x}) = k_i$

One standard model is to order the rules.

Such ordered rule sets have also been referred to as ***decision lists***.

Regression by rule induction

An algorithm: Swap-1, which learns decision rules in Disjunctive Normal Form (DNF).

```
[1]   Input:  $D$ , a set of training cases
[2]   Initialize  $R_1 \leftarrow$  empty set,  $k \leftarrow 1$ , and  $C_1 \leftarrow D$ 
[3]   repeat
[4]       create a rule  $B$  with a randomly chosen attribute as its left-hand side
[5]       while ( $B$  is not 100-percent predictive) do
[6]           make single best swap for any component of  $B$ , including
[7]               deletion of the component, using cases in  $C_k$ 
[8]           If no swap is found, add the single best component to  $B$ 
[9]       endwhile
[10]       $P_k \leftarrow$  rule  $B$  that is now 100-percent predictive
[11]       $E_k \leftarrow$  cases in  $C$  that satisfy the single-best-rule  $P_k$ 
[12]       $R_{k+1} \leftarrow R_k \cup \{P_k\}$ 
[13]       $C_{k+1} \leftarrow C_k - \{E_k\}$ 
[14]       $k \leftarrow k + 1$ 
[15]   until ( $C_k$  is empty)
[16]   find rule  $r$  in  $R_k$  that can be deleted without affecting performance on cases
[17]       in training set  $D$ 
[18]   while ( $r$  can be found)
[19]        $R_{k+1} \leftarrow R_k - \{r\}$ 
[20]        $k \leftarrow k + 1$ 
[21]   endwhile
[22]   output  $R_k$  and halt.
```

Regression by rule induction

Example of swapping rule components

Step	Predictive value (%)	Rule
1	31	p3
2	36	p6
3	48	p6 & p1
4	49	p4 & p1
5	69	p4 & p1 & p2
6	80	p4 & p1 & p2 & p5
7	100	p3 & p1 & p2 & p5

“An overview of regression techniques for knowledge Discovery”, The Knowledge Engineering Review, Vol. 14:4, 1999, 319-340

Projection pursuit regression

One problem with most local averaging techniques, such as the nearest-neighbour technique, is the *curse of dimensionality*.

Soderstrom and P. Stoica, "System Identification", Prentice Hall, New York, 1989.

Projection Pursuit Regression (PPR) forms the estimation model by reflecting the training set onto lower dimensional projections as a solution for high dimensional data sets.

Projection pursuit regression

The model approximated by the PPR algorithm is the sum of the smooth functions S of the linear projections, determined in each iteration:

$$\varphi(x) = \sum_{m=1}^M S_{\beta_m}(\beta_m \cdot X)$$

β_i the parameter vector (projection)

X is the training set against predictor variables

S_{β_m} is the smooth function and M is the number of term in the model

Projection pursuit regression

The fitting criterion is the fraction of unexplained variance that is explained by smooth S_β

$$I(\boldsymbol{\beta}) = 1 - \sum_{i=1}^n (r_i - S_\beta(\boldsymbol{\beta} \cdot \mathbf{x}_i))^2 / \sum_{i=1}^n r_i^2$$

r_i is the residual

The coefficient β that maximizes $I(\beta)$ is the optimal solution

Multivariate adaptive regression splines

A fundamental drawback of recursive partitioning regression (CART) is the lack of continuity, which affects the accuracy.

Another problem with that method is its inability to provide good approximations to some functions, even to the most simple linear ones.

Multivariate Adaptive Regression Splines (MARS) addresses these two problems of recursive partitioning regression to increase accuracy.

Multivariate adaptive regression splines

MARS builds models of the form

$$\hat{f}(x) = \sum_{i=1}^k c_i B_i(x).$$

The model is a weighted sum of basis functions $B_i(x)$

Each c_i is a constant coefficient.

Multivariate adaptive regression splines

Each basis function $B_i(x)$ takes one of the following three forms:

1. A constant.
2. A hinge function.
3. A product of two or more hinge functions..

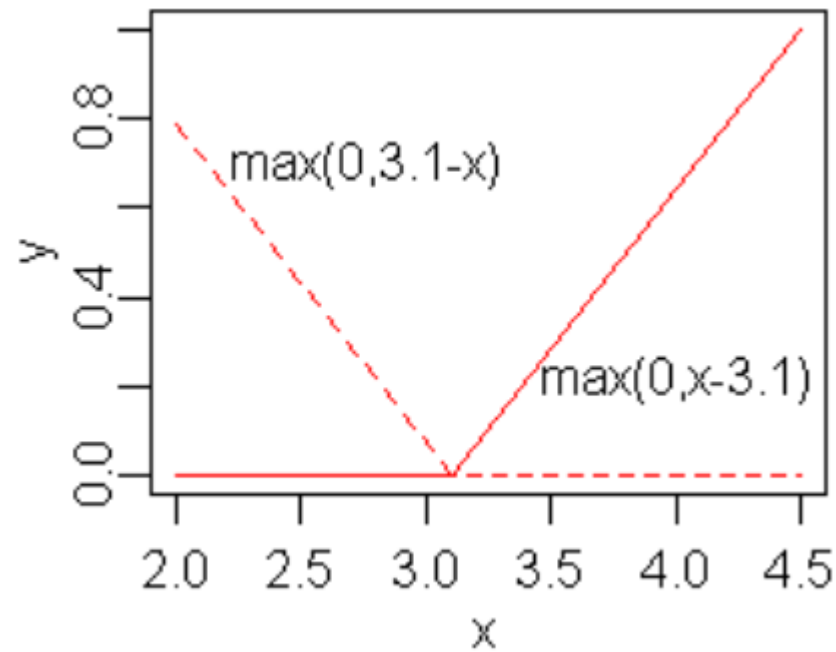
Multivariate adaptive regression splines

A hinge function takes the form

$$\max(0, x - c)$$

Or

$$\max(0, c - x)$$



Lectures by the students

Deep learning in solving regression problems

- Technical challenges.
- An application.

Support vector machines for regression.

- Technical challenges.
- <https://doi.org/10.1016/j.jclepro.2018.07.164>

Adaboost to regression problems.

- <http://dx.doi.org/10.5430/air.v3n4p64>